NDC END-SEM LAB

Exam-Ready Code Guide

Python networking programs • NS-2 Tcl simulation • Cisco CLI • Wireshark filters

Included: TCP client/server, UDP client/server, Distance Vector Routing, Dijkstra shortest path, web-cache demonstration, TCP echo client/server, NS-2 two-node simulation, Cisco configuration commands, Wireshark filters, execution instructions and sample outputs.

Use the exact algorithm and format required by your lab manual if it differs from these examples.

1. Networking commands

Windows: ipconfig, ipconfig /all, ping 8.8.8.8, tracert example.com, nslookup example.com, netstat -an, arp -a, hostname.

Linux: ip addr, ip route, ping -c 4 8.8.8.8, traceroute example.com, ss -tuln, ip neigh.

Remember: ping tests reachability; tracert/traceroute shows hops; nslookup queries DNS; arp -a displays the ARP cache; netstat/ss shows socket and connection information.

2. TCP socket programming

File: tcp_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print("TCP server waiting for a client...")

    conn, addr = server.accept()
    with conn:
        print("Connected by:", addr)
        data = conn.recv(1024).decode()
        print("Client says:", data)
        conn.sendall("Hello from TCP server!".encode())
```

File: tcp_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    client.sendall("Hello from TCP client!".encode())
    reply = client.recv(1024).decode()
    print("Server says:", reply)
```

Run: Open two terminals in the same folder. First run `python tcp_server.py`, then run `python tcp_client.py`. Use `python3` if required.

Sample client output: Server says: Hello from TCP server! The server prints the client's message and the connected address (the ephemeral port varies).

Viva: TCP is connection-oriented and reliable; AF_INET means IPv4; SOCK_STREAM selects TCP; bind associates address/port; listen waits for connections; accept accepts a connection; encode/decode convert between strings and bytes.

3. UDP socket programming

File: udp_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
    server.bind((HOST, PORT))
    print("UDP server waiting for a message...")
    data, addr = server.recvfrom(1024)
    print("Client says:", data.decode())
    server.sendto("Hello from UDP server!".encode(), addr)
```

File: udp_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client:
    client.sendto("Hello from UDP client!".encode(), (HOST, PORT))
    data, addr = client.recvfrom(1024)
    print("Server says:", data.decode())
```

Run: Start the server first in one terminal, then the client in another. Client output: Server says: Hello from UDP server!

Viva: UDP is connectionless and does not guarantee delivery/order. SOCK_DGRAM selects UDP. sendto() sends a datagram to an address; recvfrom() receives a datagram and sender address.

4. Distance Vector Routing

File: distance_vector.py

```
INF = float("inf")

# Link-cost matrix for routers A, B, C
cost = [
    [0, 1, 4],
    [1, 0, 2],
    [4, 2, 0]
]

n = len(cost)
dist = [row[:] for row in cost]
next_hop = [[None] * n for _ in range(n)]

for i in range(n):
    for j in range(n):
        if i != j and cost[i][j] != INF:
            next_hop[i][j] = j

# Repeatedly exchange distance estimates
for _ in range(n - 1):
    changed = False
    for i in range(n):
        for neighbor in range(n):
            if cost[i][neighbor] == INF:
                continue
            for dest in range(n):
                new_cost = cost[i][neighbor] + dist[neighbor][dest]
                if new_cost < dist[i][dest]:
                    dist[i][dest] = new_cost
                    next_hop[i][dest] = neighbor
                    changed = True
    if not changed:
        break

names = ["A", "B", "C"]
for i in range(n):
    print(f"\nRouting table for router {names[i]}")
    print("Destination  Cost  Next hop")
    for j in range(n):
        hop = "-" if i == j else names[next_hop[i][j]]
        print(f"{names[j]:<12} {dist[i][j]:<5} {hop}")
```

Expected costs: A→B = 1; B→C = 2; A→C = 3 via B (instead of the direct cost 4). This is an educational distance-vector-style iterative update, not a complete production routing protocol.

Viva: Distance Vector uses the Bellman–Ford update rule: D■(y) = min■[c(x,v) + D■(y)]. A routing table stores destination and forwarding information.

5. Dijkstra shortest-path algorithm

File: dijkstra.py

```
import heapq

graph = {
    "A": {"B": 1, "C": 4},
```

```
    "B": {"A": 1, "C": 2, "D": 5},
    "C": {"A": 4, "B": 2, "D": 1},
    "D": {"B": 5, "C": 1}
}

source = "A"
dist = {node: float("inf") for node in graph}
previous = {node: None for node in graph}
dist[source] = 0
pq = [(0, source)]

while pq:
    current_dist, u = heapq.heappop(pq)
    if current_dist != dist[u]:
        continue
    for v, weight in graph[u].items():
        new_dist = current_dist + weight
        if new_dist < dist[v]:
            dist[v] = new_dist
            previous[v] = u
            heapq.heappush(pq, (new_dist, v))

for node in graph:
    path = []
    current = node
    while current is not None:
        path.append(current)
        current = previous[current]
    path.reverse()
    print(f"A to {node}: cost={dist[node]}, path={' -> '.join(path)}")
```

Expected output: A to A cost 0; A to B cost 1 via A→B; A to C cost 3 via A→B→C; A to D cost 4 via A→B→C→D.

Viva: Dijkstra finds shortest paths when edge weights are non-negative. It commonly uses a priority queue.

6. Web cache demonstration

File: web_cache.py

```
cache = {}

def fetch_from_server(url):
    print("Fetching from origin server:", url)
    return f"Content of {url}"

def get_page(url):
    if url in cache:
        print("Cache HIT:", url)
        return cache[url]
    print("Cache MISS:", url)
    content = fetch_from_server(url)
    cache[url] = content
    return content

url = "https://example.com/page"
print(get_page(url))
print(get_page(url))
```

Run: python web_cache.py. First request is a cache miss and invokes the simulated origin fetch; second request is a cache hit. This is a demonstration only, not a real HTTP proxy.

Viva: A cache hit means the item is stored; a cache miss means it must be fetched. Caching can reduce latency, bandwidth use, and origin-server load. Cache invalidation determines when stored content is stale.

7. Client-server echo application

File: echo_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5002

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
```

```
    print("Echo server ready")
    conn, addr = server.accept()
    with conn:
        print("Client connected:", addr)
        data = conn.recv(1024)
        conn.sendall(data)
        print("Echoed:", data.decode())
```

File: echo_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5002

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    message = "Networking lab"
    client.sendall(message.encode())
    reply = client.recv(1024).decode()
    print("Server response:", reply)
```

Run: Start echo_server.py first, then echo_client.py. Client output: Server response: Networking lab. The example handles one client and one message.

Viva: A client initiates a request; a server waits and responds. TCP is a byte stream, so larger applications need message framing and loops to handle partial reads.

8. NS-2 Tcl simulation

NS-2 uses OTcl/Tcl scripts for simulation setup and C++ for many protocol implementations. NAM visualises recorded events. Save as sample.tcl on a machine with NS-2 installed.

```
# Create simulator and trace files
set ns [new Simulator]
set tr [open out.tr w]
$ns trace-all $tr
set nf [open out.nam w]
$ns namtrace-all $nf

# Create two nodes and a bidirectional link
set n0 [$ns node]
set n1 [$ns node]
$ns duplex-link $n0 $n1 1Mb 10ms DropTail

# UDP sender and Null receiver
set udp [new Agent/UDP]
$ns attach-agent $n0 $udp
set null [new Agent/Null]
$ns attach-agent $n1 $null
$ns connect $udp $null

# CBR traffic
set cbr [new Application/Traffic/CBR]
$cbr set packetSize_ 512
$cbr set interval_ 0.05
$cbr attach-agent $udp

$ns at 0.5 "$cbr start"
$ns at 4.5 "$cbr stop"

proc finish {} {
    global ns tr nf
    $ns flush-trace
    close $tr
    close $nf
    exit 0
}

$ns at 5.0 "finish"
$ns run
```

Execute: ns sample.tcl; then run nam out.nam to open the animation. The expected files are out.tr and out.nam.

Viva: NS-2 = Network Simulator 2; NAM = Network Animator; CBR = Constant Bit Rate; DropTail drops arriving packets when a queue is full; Agent/Null is a sink; $ns at schedules an event; $ns run starts simulation; 1Mb is link bandwidth and 10ms is link delay.

9. Cisco Packet Tracer commands

Two PCs on one switch

Example PC addresses: PC0 192.168.1.10/24 and PC1 192.168.1.20/24. A default gateway is not needed for communication within this subnet. Test from PC0: ping 192.168.1.20.

```
Switch> enable
Switch# show mac address-table
Switch# show interfaces status
```

Router with two directly connected LANs

Example: G0/0 = 192.168.1.1/24 and G0/1 = 192.168.2.1/24. Set each PC's gateway to its local router interface. Interface names vary by device model.

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 192.168.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 192.168.2.1 255.255.255.0
 no shutdown
exit
end
show ip interface brief
show ip route
```

Static routes between two routers

```
! On R1: next hop toward 192.168.2.0/24 is 10.0.0.2
ip route 192.168.2.0 255.255.255.0 10.0.0.2

! On R2: next hop toward 192.168.1.0/24 is 10.0.0.1
ip route 192.168.1.0 255.255.255.0 10.0.0.1
```

These static route examples assume the 10.0.0.0 point-to-point network and router interfaces are already configured and reachable.

Useful checks: show running-config; show ip interface brief; show ip route; show interfaces; ping . C = connected route; S = static route in typical Cisco IPv4 routing output.

10. Wireshark filters and what to inspect

Display filter	Purpose
tcp	Show TCP packets
udp	Show UDP packets
icmp	Show ICMPv4 packets
dns	Show DNS packets
http	Show dissected HTTP traffic when visible
ip.addr == 192.168.1.10	Packets to or from this IPv4 address
tcp.port == 80	TCP packets using port 80
udp.port == 53	UDP packets using port 53
tcp.flags.syn == 1	TCP packets with SYN flag set
tcp.flags.syn == 1 && tcp.flags.ack == 1	TCP SYN-ACK packets
dns.qry.name == "example.com"	DNS queries for example.com

Practical workflow: Select the active interface → start capture → generate traffic (for example, ping a host or run nslookup example.com) → enter a display filter → select a packet → inspect source/destination IP, protocol, ports, flags and packet details.

Viva: Capture filters limit packets collected; display filters limit packets shown after capture. ICMPv4 Echo Request is type 8 and Echo Reply is type 0. TCP's handshake is SYN, SYN-ACK, ACK. HTTPS payloads are encrypted unless suitable decryption is configured.

11. Performance formulas

Throughput: successfully received data / elapsed time. For bits per second, use bits and seconds.

End-to-end delay: reception time – transmission time.

Packet Delivery Ratio (PDR): (packets received / packets sent) × 100%.

Packet loss: packets sent – packets received, assuming the counts refer to the same flow and observation period.

12. Final rapid viva revision

Q. What is TCP?
Connection-oriented, reliable, ordered byte-stream transport.

Q. What is UDP?
Connectionless datagram transport without delivery/order guarantees.

Q. What is a routing table?
Information used to choose where to forward packets.

Q. What is a cache hit?
The requested item is already cached.

Q. What is a switch?
A Layer 2 device that forwards Ethernet frames using MAC addresses.

Q. What is a router?
A Layer 3 device that forwards packets between IP networks.

Q. What is ARP?
A protocol for resolving local IPv4 addresses to link-layer MAC addresses.

Q. What is DNS?
A naming system that maps domain names to records such as IP addresses.

Q. What is NAM?
Network Animator for visualising NS-2 simulation events.

Q. What is CBR?
Constant Bit Rate traffic generation model.